

Assignment 2 (20%), 2026

- This assignment is due on Sunday 27th September, 2026 at 11:59pm, to be submitted using Canvas (see instructions below).
- This assignment contributes 20% to your final mark.
- Late assignments will be deducted 5% (5 marks out of a possible 100) for each day late, starting on the day the assignment is due and including weekends. Assignments submitted more than 10 days late will receive zero.
- Any special consideration requires you to go through the Special Consideration form process through Sydney Student.
- Incidences of academic dishonesty or plagiarism will be referred to the academic honesty coordinator and could result in a zero mark being awarded for this assignment, or automatic failure of the entire subject.
- This assignment should take the average student 12 hours to complete.

Objectives

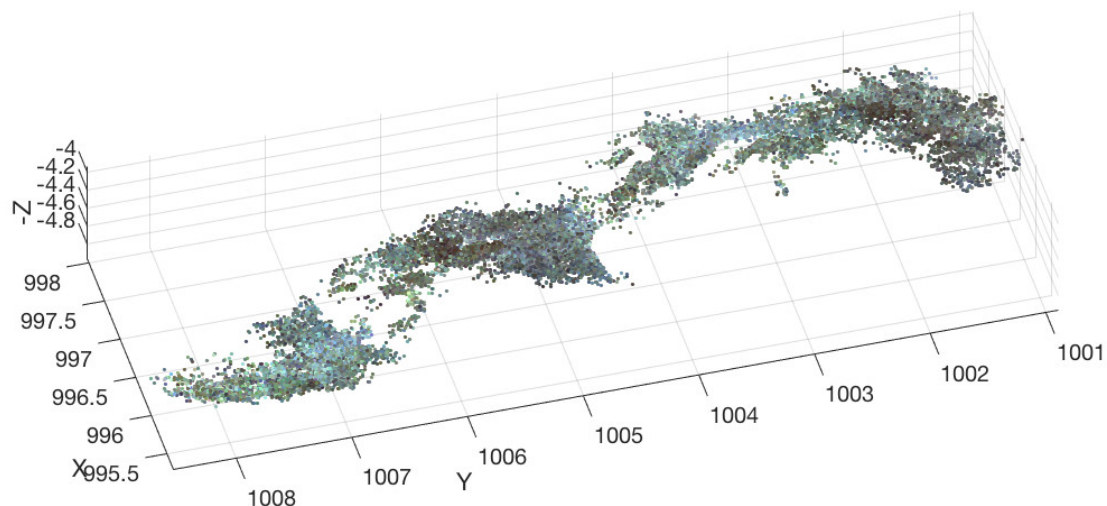
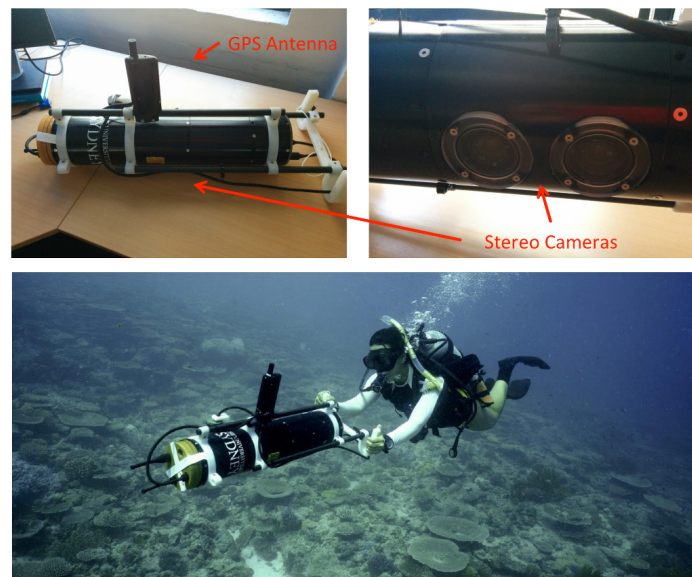
- In this assignment, you will implement and test algorithms for stereo vision and image classification
- For each of these problems, you will develop python code that implements your algorithms. You will write a report examining existing approaches to your problem, detailing your design process, and presenting and examining your results using the provided image datasets.

Submission

- You will provide one report file for the assignment. The report will include the Sections “Question 1”, “Question 2” and an appendix with subsections for each question. Each of the first two sections will contain the sub-sections “Introduction”, “Methodology” and “Results and Discussion”.
- For each question you will submit **working** python code. This can either be in a Jupyter notebook or standalone Python code (for each question make sure there is a script file named “mainQN.py” where N is the question number). This script file will be run by the tutors during marking and should run all of your code and produce all of your plots for the questions.
- There are TWO submission portals for this assignment, one for the report and one for your python code, both via Turnitin on the course Canvas site:
 - The report should be submitted to the “**Report Submission**” portal
 - Your python code is to be zipped into a single file (i.e. “**SID_Assignment2.zip**” where **SID** is you student number) and submitted via the “**Code Submission**” portal.
 - You will need to submit both your report AND python code to receive marks for the assignment.
- All python code must be commented so that the code can be read and understood without the aid of the report. If your code does not work, you will receive **0** marks for the code section of the marking unless you provide an explanation in the report as to why your code didn’t work.

- Acknowledging use of AI Tools: If you use generative AI tools to assist in working on your assignment (for example LLMs such as ChatGPT, Claude, Microsoft Co-pilot etc.), you are required to acknowledge it: this includes acknowledging any tools that use generative AI, such as translation tools, paraphrasing tools or referencing tools:
 - Include a statement at the end of your report acknowledging any tools you have used, and how you have used them to assist
 - Allowable uses include suggestions for paraphrasing and use of language in reports, feedback on report structure and presentation, suggestions for code in your implementation of experiments etc.
 - Remember that you are ultimately responsible for the work you present and submit: don't use AI tools to subvert your own learning
 - For more details on responsible AI tool use in assessments, see: <https://www.sydney.edu.au/students/responsible-ai-use.html>

Question 1: 3D Reconstruction using Underwater Stereo Vision (50%)



- In this question, you will work with a dataset of stereo images captured using an underwater diver-operated stereo camera rig (the “diver-rig”, see above figure) over a shallow water coral reef.
- The “diver-rig” contains a downwards-facing stereo pair with one colour camera (left) and one monochrome camera (right). Both cameras are placed in a waterproof housing. The “diver-rig” also carries a combination of other sensors (GPS, magnetic heading and tilt sensing) that allow for the position and orientation of the stereo camera pair to be measured at each camera frame (i.e. the rotation matrix and translation vector representing the left camera coordinate system relative to a world-fixed coordinate system). Processing of the stereo imagery allows one to construct a 3D model of the reef using collected imagery and sensor data.
- On the Canvas site, download and unzip “assignment2_stereodata.zip”. This file contains a collection of 49 stereo image pairs and three python-pickle files that contain the calibration parameters of the stereo camera system, the camera trajectory parameters for each captured stereo pair and a reference terrain

model, generated from a larger series of images and other sensor measurements. Left camera images are stored in the folder “images_left” and right camera images in the folder “images_right”. The file “stereo_calib_diver.pkl” contains the stereo camera calibration parameters. The file “camera_pose_data.pkl” contains a python dictionary containing:

- **R**: a 3 x 3 x 49 array of rotation matrices, one 3 x 3 matrix for each of the 49 camera images (see week 5 slides for the definition of these parameters)
- **t**: a 3 x 49 array of translation vectors, one 3 x 1 vector for each of the 49 camera images (see week 5 slides for the definition of these parameters)
- **filenams_left**: a list corresponding left camera filenames
- **filenames_right**: a list corresponding right camera filenames
- The file “terrain_data.pkl” contains reference terrain data in three variables:
 - **height_grid**: a 2D array of terrain height values
 - **X**: a vector of X-position values, corresponding to the rows of height_grid
 - **Y**: a vector of Y-position values, corresponding to the columns of height_grid
 - You can produce a plot of the reference terrain using:

```
X, Y = np.meshgrid(np.flip(terrain_data['X']), terrain_data['Y'])
surf = ax.plot_surface(X, Y, -terrain_data['height_grid'], rstride=5, cstride=5)
```

- Use the stereo pairs and corresponding information on the rotation and translation of the left camera with respect to a fixed-world reference system for each of the 49 image pairs to build a 3D point cloud of the underwater terrain, corresponding to the surfaces seen in the camera images:
 - For each stereo pair, extract interest points of your choice (i.e. ORB, SIFT etc.) and match these across the left and right images, making sure to use an outlier-rejection process to remove bad matches
 - Use these matched feature points to produce a set of corresponding 3D points in space with respect to the left camera’s frame of reference, for each of the 49 stereo pairs
 - Produce a single point cloud in the world reference frame using the point clouds from each of the 49 stereo pairs. For each set of points, translate and rotate the points into the world reference frame using the camera rotation and translation data contained in “camera_pose_data.pkl”.
 - Compare your reconstructed point cloud to the provided reference terrain model
- You should provide a python script in your code submission called “**MainQ1.py**” which when run, implements your stereo vision process algorithms and produces a plot of the reconstructed point cloud. You do not need to include the stereo images themselves in your code submission. At the top of your script have two python variables `images_left_dir` and `images_right_dir` which represent the path to the folder containing the left and right image sets; these will be changed by the tutor based on the computer that your code is run on.

- In your report, under “Question 1”, detail your approach to producing the point cloud and present your results. This section of the report (Question 1) should have the following subsections:
 - **Introduction:** Describe how stereo vision works and from your own research some of the applications in which stereo vision has been applied and the algorithms used.
 - **Methodology:** Describe your approach to producing a 3D point cloud of the imaged terrain from stereo image pairs, calibration data and camera pose information.
 - **Results and Discussion:** Use plots from your python code to discuss your results, focussing on the feature extraction and matching process for image pairs and the final 3D point cloud reconstruction.
 - This section of the report has a **4-page limit** (not including the appendix).

Question 1 Marking Scheme

Code Implementation (20/50)	Code doesn't work	0 Marks awarded for code
	Code works, is readable and commented and the implementation is correct.	/20 Marks
Report (30/50)	Introduction: Describe how stereo vision works and from your own research some of the applications in which stereo vision has been applied and the algorithms used.	/5 Marks
	Methodology: Have features been successfully extracted and matched from image pairs? Has the final world-referenced point cloud been constructed correctly using the individual triangulations for each stereo pair?	/10 Marks
	Discussion and Analysis: Have you analysed and compared the performance of your feature extraction and matching for different types of interest points and for different feature extraction parameters? What effect does this have on the final accuracy, density and types of structures covered in the final point cloud?	/15 Marks

Question 2: Scene Classification (50%)



1. Desert
2. Sky
3. Ocean
4. Urban
5. Park



1. Park
2. Road
3. Urban
4. Sky
5. Snow

Scene classification refers to assigning a semantic label that describe the context of the environment that is pictured in a single image and is a form of image classification. In this question, your task is to develop a machine learning algorithm that can perform scene classification based on RGB images. You must take a systematic approach to the development of the algorithm that involves justifying the choice of data pre-processing, feature extraction, model training, validation and testing, and parameter selection.

In order to train and test your algorithm, download:

- dataset.zip – in this folder there are several subfolders containing the images. The labels for the images within each subfolder are obtained from the folder name. e.g. img_000001.jpg within folder ‘desert’ is an image whose ground truth label is ‘desert’.

This dataset is a subset of the much larger ‘places’ dataset [1], however no other data apart from that given in dataset.zip can be used.

- The algorithm you generate should take as an input an RGB image and output the predicted scene label. Optionally, you may also output a set of labels in order of confidence. This is useful when assessing performance based on top-k accuracy.
- Your algorithm should be implemented in python as a function named `assign2_sceneclassifier` within a file “assign2_sceneclassifier.py”. This function should use the prototype:

```
[class_labels] = assign2_sceneclassifier(image_path)
```

- Where `image_path` is a path to an image file and `class_labels` is a 1 x N list of strings representing the assigned label for the scene contained in the image, in order of the most likely class. The strings contained in the array should match the folder names. If your algorithm makes a single decision of the scene class, then N would be equal to 1.
- You may have any other source files within the same directory as “assign2_sceneclassifier.py” and also don’t forget to include your pre-trained model weights file in the directory too.

Report

In your report, under “Question 2”, detail your design choices and provide an empirical validation of your model selection. Your report must cover all aspects of the algorithm including:

- Pre-processing of the data – including (but not limited to) choice of colour space, image quantisation, histogram equalisation etc.
- Feature extraction method – techniques include (but not limited to) intensity, colour or texture histograms, feature points etc. Describe why you are selecting a particular method and what is your intuition behind why it will work well.
- Algorithm selection: you may choose to experiment with different types of classification algorithms. At a minimum, you must implement either a KNN or SVM classifier.
- Model training – split the data into training and testing sets and perform cross validation and report the results. Decide on which metrics are useful to justify your algorithm selection such as accuracy, precision, recall, F1-score, top-K accuracy etc.
- parameter selection – models such as KNNs, SVMs etc. have underlying parameters that need to be selected e.g. number of nearest neighbours, kernel types and model structure. Show the justification for your choice of parameters in your model.
- The report has a **4-page limit** excluding references and any appendices that might be included.

Question 2 Marking Scheme

Code (10/50)	Code doesn't work	0 Marks awarded for code
	Code works, is readable and commented.	/10 Marks
Report (40/50)	Introduction: General description of scene classification and in what applications algorithms for performing scene classification is useful. Briefly describe the types of algorithms that have previously been used.	/5 Marks
	Methodology: have the pre-processing and feature extraction methods been explained and justified? Has a proper experimental methodology been used to determine the final proposed model? Are the metrics suitable for the purpose of scene classification?	/25 Marks
	Discussion and Analysis: Is there an explanation for the miss-classification between different classes or why certain features work well? How do you think it compares with state-of-the-art techniques and why?	/10 Marks

References:

[1] *Places: A 10 million Image Database for Scene Recognition*, B. Zhou, A. Lapedriza, A. Khosla, A. Oliva, and A. Torralba, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017.